

Apziva: MonReader - Computer Vision Project Guide

The problem is simple to state: given a single image from a page-flipping video, predict whether the page is being flipped or not. Binary classification.

But the interesting part is not only performance (F1score). It is just a constraint. You need F1 above 0.91 AND the model has to fit under 40 MB, because it will be shipped to a mobile app. That constraint is what makes this a real-world CV project instead of a simple Kaggle exercise.

Section 1: Prerequisite Refresher (Week 0)

You did the DL basics before the NLP project, so most of this should feel familiar. Do a quick brush-up if it has been a while.

1.1 Quick DL Refresher

Topic	Resource	Time
Neural networks intuition, backprop, gradient descent	Andrej Karpathy, Neural Networks Zero to Hero, Part 1 (micrograd) on YouTube	2.5 hrs
Keras and TensorFlow quickstart	Official TensorFlow "Beginner Quickstart" and "Keras Sequential API" tutorial pages	2 hrs
Convolution operation, intuitively	3Blue1Brown, "But what is a convolution?" on YouTube	30 min

We are using Keras/TensorFlow for this project. It is much easier than PyTorch for transfer learning experiments, and pretrained weights come baked in.

Section 2: Core CV Learning Path (Weeks 1 and 2)

The backbone is Andrew Ng's Deep Learning Specialization.

2.1 Andrew Ng Deep Learning Specialization

Course	What it covers	Priority
Course 1: Neural Networks and Deep Learning	Perceptrons, forward and back prop, shallow and deep nets	Mandatory
Course 2: Improving Deep Neural Networks	Regularization, dropout, batch norm, optimizers, hyperparameter tuning	Mandatory
Course 3: Structuring ML Projects	Error analysis, train/dev/test splits, orthogonalization	Optional
Course 4: Convolutional Neural Networks	Convolutions, pooling, LeNet, AlexNet, VGG, ResNet, Inception, MobileNet, transfer learning, object detection basics	Mandatory (most important for this project)
Course 5: Sequence Models	RNNs, LSTMs, attention	Optional for this project, but do it for complete learning

Course 4 is the one that maps directly to your project. Take your time on it.

2.2 Supplementary Resources

Topic	Resource	Why
CNN visual intuition	CNN Explainer by Poloclub, cnn-explainer.polo.club (interactive site)	Best interactive CNN visualization on the internet
Convolutional filters, strides, padding	Deeplizard, "Convolutional Neural Networks" playlist on YouTube	Short focused videos with clean visuals
VGG, ResNet, Inception, MobileNet architectures	Aladdin Persson, "PyTorch Image Classification Architectures" playlist on YouTube (concepts still apply to Keras)	Walks through each architecture visually
Transfer learning with Keras	TensorFlow official tutorial: "Transfer learning and fine-tuning"	Exact code patterns you will use
Data augmentation	Keras Preprocessing docs and TensorFlow "Data augmentation" tutorial	Direct reference for the project
Grad-CAM interpretability	Keras official example: "Grad-CAM class activation visualization"	Copy-adapt pattern for your notebook
MobileNet depthwise separable convolutions	Any 15-min YouTube video on "Depthwise Separable Convolutions" (search on YouTube)	You need to understand why MobileNet is small
EfficientNet compound scaling	The AI Epiphany or Yannic Kilcher on YouTube, "EfficientNet paper walkthrough"	Understand the scaling philosophy

2.3 Optional: fast.ai Practical Deep Learning

fast.ai overlaps with Andrew Ng about 60 to 70 percent. The difference is style. Andrew Ng teaches bottom-up (math first, then code). fast.ai teaches top-down (train a model in lesson 1, understand internals later). Some learners love it, others find the double coverage wasteful.

My honest advice: skip it unless you finish Andrew Ng early and want a different perspective. Do not do both from start to finish, you will burn out.

2.4 Breadth Topics You Should Know Exist

Watch a short video or read a quick blog on each. You will not use these in the project.

Topic	Resource
Object detection (YOLO, Faster R-CNN)	Aladdin Persson, "YOLOv1 from scratch" intro video on YouTube
Semantic segmentation (U-Net)	Any 15 min YouTube video on U-Net
Vision Transformers (ViT)	Yannic Kilcher, "An Image is Worth 16x16 Words" on YouTube
Image generation (GANs, Diffusion)	Any 20 min overview video, no need to go deep
OCR (relates to MonReader's bigger pipeline)	Tesseract OCR overview blog

Section 3: Architecture Deep Dive

Since you will be comparing five architectures, spend focused time understanding each one before writing code. This ordering follows how CNNs evolved.

#	Architecture	Year	What it introduced	Params (approx)
1	LeNet-5	1998	First working CNN, conv + pool + FC	60K
2	AlexNet	2012	ReLU, dropout, GPU training, deep CNN wins ImageNet	60M
3	VGG16	2014	Uniform 3x3 convs stacked deep, simple design	138M
4	Inception (GoogLeNet)	2014	Parallel filters of different sizes, 1x1 convs for reduction	6M
5	ResNet50	2015	Skip connections solve vanishing gradients, enables very deep nets	25M

#	Architecture	Year	What it introduced	Params (approx)
6	MobileNetV2	2018	Depthwise separable convs, inverted residuals, made for mobile	3.4M
7	EfficientNetB0	2019	Compound scaling of depth, width, resolution	5.3M

Look at that params column and read it top to bottom. This is why MobileNet exists. You are going to feel this pain personally when you check your model file sizes.

Section 4: Project Execution (Weeks 3 and 4)

Now the hands-on part. Follow the steps in order. Everything goes in a single consolidated notebook with a final comparison table.

4.1 Project Setup

Same cookiecutter-data-science template you used for NLP.

- data/raw/, data/processed/
- notebooks/ (single main notebook, plus a scratch notebook if you want)
- src/ for reusable code (data loaders, training loops, evaluation helpers)
- models/ for saved model files
- reports/ for figures and blog draft

Create the GitHub repo on day one. Update the README as you go. Use Google Colab with GPU runtime (free T4). Save model files to Google Drive so you do not lose them when the session dies.

4.2 Step by Step Tasks

Step	Task	Notes
1	EDA. Count images per class in training and testing. Plot a few sample images from each class. Check image resolution, aspect ratios, and any obvious corruption.	You should see class balance or imbalance early.
2	Data loading pipeline. Use <code>tf.keras.preprocessing.image_dataset_from_directory</code> or <code>ImageDataGenerator</code> . Set image size (start with 224x224 for ImageNet models). Batch size 32.	Set a validation split from training folder, say 80/20.
3	Data augmentation setup. Random flip, rotation, zoom, contrast. Keep it mild first.	Do not augment the test set.

Step	Task	Notes
4	Model 1: Custom CNN from scratch. Build 3 to 5 conv blocks (Conv2D, BatchNorm, ReLU, MaxPool). Add a small FC head. Train from random init.	This is where you play with filters, strides, activations. Try 32, 64, 128, 256 filter progressions.
5	Model 2: VGG16 with feedforward head. Load VGG16 with imagenet weights, freeze base, add GAP + Dense + Dropout + Dense(1, sigmoid). Train.	No augmentation in this variant.
6	Model 3: VGG16 with data augmentation. Same as Model 2 but with the augmentation layer active.	Now you see the effect of augmentation cleanly.
7	Model 4: ResNet50 with imagenet weights, frozen base, custom head. Train.	Compare training speed and F1 vs VGG16.
8	Model 5: EfficientNetB0 with imagenet weights, frozen base, custom head. Train.	Note the smaller size and comparable performance.
9	Model 6: MobileNetV2 with imagenet weights, frozen base, custom head. Train.	This should be your final winner on size, and hopefully very close on F1.
10	Fine-tuning pass. For the top 2 or 3 models, unfreeze the last few layers of the base and train with a very low learning rate (1e-5).	This usually gives a meaningful F1 bump.
11	Evaluation. For each model, compute F1, precision, recall, accuracy, ROC-AUC, PR-AUC on the test set. Plot confusion matrix.	Put everything into a single comparison table.
12	Model size analysis. Save each model as .h5 or SavedModel. Record file size in MB.	Add a Size column to your comparison table.
13	Grad-CAM interpretability. Pick your best model. Generate Grad-CAM heatmaps on 5-10 test images from each class.	This tells you whether the model is looking at the actual page-flip motion or spurious features. Great blog material.
14	Final model selection. Pick the model that best balances F1 above 0.91 AND size under 40 MB. Justify your choice in the report.	Usually this is MobileNetV2 or fine-tuned MobileNetV2.
15	(Optional) Model compression. Try quantization (post-training float16 or int8 via TFLite converter). Try weight pruning via tensorflow_model_optimization. Measure size and F1 after each.	Only if you have time. Great blog talking point.
16	Gradio or Streamlit app. Upload an image, get back "Flipping" or "Not Flipping" with the confidence score. Add a Grad-CAM overlay panel if you want to level up.	Host it free on Hugging Face Spaces.
17	Write the blog on Medium or Hashnode. Walk through the architecture comparison, the size vs F1 tradeoff, the Grad-CAM findings, and why MobileNet won.	Include the comparison table and app screenshots.
18	Polish the GitHub README. Problem statement, approach, final comparison table, model choice justification, how to run, app demo link, blog link. Optional: record a short video demo of the app and embed the link.	This is your portfolio piece.

4.3 What the Final Comparison Table Should Look Like

Build one table like this in your notebook and paste it into your blog and README:

Model	Params	Size (MB)	F1	Precision	Recall	ROC-AUC	PR-AUC
Custom CNN	?	?	?	?	?	?	?
VGG16 + FF	?	?	?	?	?	?	?
VGG16 + Aug	?	?	?	?	?	?	?
ResNet50	?	?	?	?	?	?	?
EfficientNetB0	?	?	?	?	?	?	?
MobileNetV2	?	?	?	?	?	?	?
MobileNetV2 (fine-tuned)	?	?	?	?	?	?	?

This one table is the entire story of your project.

4.4 The Key Insight You Should Walk Away With

By the end of this project you should be able to say this out loud without stumbling:

"For pure accuracy, ResNet50 and EfficientNetB0 are strong. But on an edge device, MobileNetV2 wins because it uses depthwise separable convolutions, which massively cut parameter count and computation while keeping F1 within 1 or 2 points of the larger models. When your constraint is on-device size, MobileNet is not a fallback, it is the right answer."

If you can say that and back it up with your comparison table, you have understood the point of this project.

Section 5: Practical Tips and Common Pitfalls

Read this twice. These trip up almost every mentee of mine.

- GPU on Colab. Runtime, Change runtime type, GPU. Otherwise training will crawl.
- Do not preprocess twice. Each pretrained model has its own preprocess_input function (VGG16, ResNet50, EfficientNet, MobileNet all differ). Use `tf.keras.applications.<model>.preprocess_input`, do not manually divide by 255 on top of it.

- Freeze first, fine-tune later. Always train with the base frozen first for a few epochs to warm up the head. Then unfreeze and fine-tune with a low LR. If you unfreeze from the start, the pretrained weights get wrecked.
 - Use GlobalAveragePooling2D, not Flatten. Flatten blows up the parameter count of the head. GAP keeps it small, which matters for your size budget.
 - Use Dropout in the head. 0.3 to 0.5 is a good starting range.
 - Class balance. If flip vs not flip is imbalanced, use `class_weight` in `model.fit`, or oversample the minority class.
 - Callbacks. Use `EarlyStopping` on val loss (patience 5), `ReduceLROnPlateau`, and `ModelCheckpoint`. Do not train blind.
 - Random seeds. Set numpy, tensorflow, and python random seeds at the top of the notebook. Reproducibility matters when you compare 6 models.
 - Save models properly. Use `model.save("path.keras")` or SavedModel format. Check the file size using `os.path.getsize` immediately after saving.
 - Grad-CAM gotcha. It needs the last conv layer name. For pretrained models this can be non-obvious. Print `model.summary()` to find it.
 - Do not chase 100 percent F1. If you see 0.99+ on both train and test, be suspicious. Look at Grad-CAM. The model might be latching onto a shortcut feature.
-

Section 6: Suggested Weekly Plan

Week	Focus	Hours
Week 0 (optional)	Quick DL and Keras refresher	4 to 5
Week 1	Andrew Ng Courses 1 and 2, plus CNN Explainer and convolution basics	14
Week 2	Andrew Ng Courses 3 and 4, plus architecture deep dives (VGG, ResNet, MobileNet, EfficientNet)	14
Week 3	Project Steps 1 to 10. EDA, data loaders, augmentation, train all 6 models	14
Week 4	Project Steps 11 to 18. Evaluation table, Grad-CAM, final model, app, blog, README	14

Section 7: When You Are Done

You are done when:

1. Your GitHub repo runs end to end.
2. Your comparison table is complete with all metrics and model sizes.

3. You have at least one model that achieves F1 above 0.91 and stays under 40 MB.
 4. Your Gradio or Streamlit app is live on Hugging Face Spaces.
 5. Your blog is published and linked in the README.
 6. You have Grad-CAM visualizations in the report.
 7. You can explain, without notes, what depthwise separable convolutions are and why MobileNet wins on edge.
-
-